

SISTEMA DE GESTÃO COMERCIAL

Assistência Técnica — TechFix

Entrega 1 — Modelagem e Arquitetura

Disciplina: Construção e Configuração de Software

Campo	Informação
Sistema	SGC — Sistema de Gestão Comercial
Contexto	Assistência Técnica (TechFix)
Entrega	1 — Modelagem e Arquitetura
Data de entrega	02/04/2025
Tecnologias	Java 21 · Spring Boot 3 · MySQL · JWT

1. Descrição do Sistema

O **SGC TechFix** é um sistema de gestão comercial para assistência técnica de pequeno porte. Contempla o atendimento a clientes que entregam equipamentos eletrônicos (computadores, notebooks, celulares e periféricos) para diagnóstico e reparo.

Problemas resolvidos: controle manual de ordens de serviço, perda de histórico de atendimentos, rastreamento de peças e estoque, e falta de visibilidade financeira. A solução integra todos esses processos em uma plataforma com interface web e API REST.

Objetivos

- Gerenciar cadastro de clientes e equipamentos
- Controlar ordens de serviço do diagnóstico até a entrega
- Gerenciar estoque de peças e insumos
- Registrar vendas de produtos e serviços
- Gerar relatórios gerenciais por período
- Controlar acesso por perfil (ADMIN e FUNCIONÁRIO)

2. Requisitos Funcionais

ID	Nome	Descrição
RF01	Autenticação	Login com username/senha retornando token JWT com expiração configurável.
RF02	Gestão de Usuários	Cadastro, edição e inativação de usuários com perfis ADMIN ou FUNCIONARIO.
RF03	Gestão de Clientes	CRUD de clientes. CPF único, e-mail válido. Não excluir cliente com vendas.
RF04	Gestão de Equipamentos	Vincular equipamentos (tipo, marca, modelo, série) a um cliente.
RF05	Gestão de Produtos	Cadastrar produtos com nome, preço e estoque. Preço ≥ 0 . Bloquear venda sem estoque.
RF06	Registro de Venda	Registrar vendas com cliente, itens e data. Valor total calculado automaticamente. Atualizar estoque.
RF07	Relatório por Período	Gerar relatório de vendas filtrando por data início e fim.
RF08	Relatório por Cliente	Listar todas as vendas de um cliente específico.
RF09	Gráfico Anual	Representação gráfica das vendas agrupadas por mês no ano selecionado.
RF10	Proteção de Rotas	Todos os endpoints (exceto /auth/login) exigem token JWT válido no header Authorization.

3. Requisitos Não Funcionais

ID	Categoria	Descrição
RNF01	Segurança	Senhas com BCrypt. JWT com expiração (padrão 8h). Rotas protegidas por perfil.
RNF02	Desempenho	Consultas respondendo em até 2 s para bases com até 10.000 registros.
RNF03	Disponibilidade	Disponibilidade mínima de 99% em ambiente de produção local.

ID	Categoria	Descrição
RNF04	Manutenibilidade	Código em camadas com responsabilidades definidas. Baixo acoplamento, alta coesão.
RNF05	Portabilidade	Backend executável em qualquer SO com JDK 21+. Banco MySQL 8+.
RNF06	Usabilidade	Interface responsiva. Tarefas principais concluíveis em no máximo 3 cliques.
RNF07	Interoperab.	Endpoints REST retornam JSON com códigos HTTP semânticos (200, 201, 400, 401, 404).
RNF08	Rastreabilidade	Toda venda registra o usuário (funcionário) responsável pelo lançamento.

4. Arquitetura Proposta

O sistema adota a **Arquitetura em Camadas**, padrão amplamente utilizado em aplicações Spring Boot. Promove separação de responsabilidades, facilita testes unitários e reduz acoplamento.

Camada	Responsabilidade	Tecnologia / Classes
Apresentação	Interface gráfica. Consome a API REST via HTTP.	Web (HTML/CSS/JS) ou Swing
Controller	Recebe requisições HTTP, valida entradas e delega ao Service.	AuthController, ClienteController, ProdutoController, VendaController
Serviço (Service)	Contém regras de negócio e orquestra repositórios.	ClienteService, ProdutoService, VendaService, AuthService
Domínio (Domain)	Define entidades e interfaces de repositório.	Cliente, Produto, Venda, ItemVenda, Usuario
Persistência (Repository)	Acesso ao banco via Spring Data JPA.	ClienteRepository, ProdutoRepository, VendaRepository
Banco de Dados	Armazenamento persistente.	MySQL 8+

Fluxo de uma requisição

Cliente HTTP → Controller (valida JWT, recebe DTO) → Service (aplica regras) → Repository (persiste/consulta) → Banco de Dados → resposta JSON.

Segurança — JWT

- Endpoint público: POST /auth/login — retorna o token JWT.
- Demais endpoints exigem header: Authorization: Bearer <token>.
- Filtro JwtAuthenticationFilter valida o token em todas as requisições.
- Controle por perfil via Spring Security: ADMIN acesso total; FUNCIONARIO restrito.

5. Padrões de Projeto

5.1 Repository Pattern — principal

Abstrai o acesso ao banco de dados. Interfaces como ClienteRepository estendem JpaRepository do Spring Data, isolando a camada de serviço dos detalhes de persistência.

- Onde: pacote domain/repository/
- Benefício: desacoplamento entre lógica de negócio e persistência. Facilita testes com mocks.

5.2 DTO Pattern — complementar

Separa os dados trafegados na API das entidades de domínio. ClienteDTO e VendaDTO controlam quais campos são expostos, evitando vazamento de dados sensíveis (ex: senha).

- Onde: pacote dto/
- Benefício: segurança e versionamento da API sem impacto no modelo interno.

5.3 Singleton — via Spring IoC

O Spring gerencia todos os beans (@Service, @Repository, @Component) como Singletons, garantindo uma única instância de cada componente no contexto da aplicação.

- Onde: todos os componentes gerenciados pelo Spring.
- Benefício: economia de memória e consistência de estado.

6. Diagrama de Domínio

Representa as entidades centrais do negócio e seus relacionamentos. O diagrama visual UML deve ser anexado em docs/diagramas/diagrama_dominio.png no repositório.

Entidade	Atributos principais	Relacionamentos
Usuario	id, username, senha, perfil (ADMIN/FUNCIONARIO), ativo	1 Usuario registra N Vendas
Cliente	id, nome, cpf, email, telefone, endereco	1 Cliente possui N Equipamentos 1 Cliente realiza N Vendas
Equipamento	id, tipo, marca, modelo, numSerie, observacao	N Equipamentos pertencem a 1 Cliente
Produto	id, nome, descricao, preco, qtdEstoque, estoqueMin	N Produtos aparecem em N Vendas via ItemVenda
Venda	id, data, valorTotal, cliente_id, usuario_id	1 Venda possui N ItensVenda 1 Venda pertence a 1 Cliente 1 Venda registrada por 1 Usuario
ItemVenda	id, quantidade, precoUnitario, venda_id, produto_id	N ItensVenda referenciam 1 Produto N ItensVenda pertencem a 1 Venda

7. Diagrama de Classes

Principais classes Java e seus membros relevantes. O diagrama visual UML deve ser anexado em docs/diagramas/diagrama_classes.png.

Classe / Interface	Pacote	Membros principais
Usuario	domain.model	id, username, senha, perfil (enum), ativo
Cliente	domain.model	id, nome, cpf, email, telefone, endereco, vendas
Produto	domain.model	id, nome, descricao, preco, qtdEstoque, ativo
Venda	domain.model	id, data, valorTotal, cliente, usuario, itens
ItemVenda	domain.model	id, produto, quantidade, precoUnitario
ClienteService	service	listar(), buscarPorId(), salvar(), atualizar(), deletar()
VendaService	service	registrar(), buscarPorId(), listarPorPeriodo(), listarPorCliente()
AuthService	service	autenticar(), gerarToken(), validarToken(), extrairUsername()
ClienteController	controller	GET /clientes, POST /clientes, PUT /clientes/{id}, DELETE /clientes/{id}
VendaController	controller	GET /vendas, POST /vendas, GET /vendas/{id}, GET /vendas/relatorio
AuthController	controller	POST /auth/login (endpoint público, retorna JWT)
ClienteRepository	domain.repository	findByCpf(), existsByVendas() — extends JpaRepository
JwtService	config	gerarToken(), validarToken(), extrairUsername()
GlobalExceptionHandler	exception	handleBusiness(), handleNotFound() — @RestControllerAdvice

8. Diagrama Lógico do Banco de Dados

Tabelas, colunas, tipos de dados e chaves do banco relacional MySQL. O diagrama visual deve ser gerado pelo MySQL Workbench e salvo em docs/diagramas/diagrama_banco.png.

Tabela	Coluna	Tipo	Restrições
usuarios	id	BIGINT	PK, AUTO_INCREMENT
	username	VARCHAR(100)	NOT NULL, UNIQUE
	senha	VARCHAR(255)	NOT NULL (BCrypt)
	perfil	ENUM	ADMIN / FUNCIONARIO
	ativo	TINYINT(1)	DEFAULT 1
clientes	id	BIGINT	PK, AUTO_INCREMENT
	nome	VARCHAR(150)	NOT NULL
	cpf	VARCHAR(14)	NOT NULL, UNIQUE
	email	VARCHAR(150)	NOT NULL, UNIQUE
	telefone	VARCHAR(20)	
	endereco	VARCHAR(255)	
	id	BIGINT	PK, AUTO_INCREMENT
	cliente_id	BIGINT	FK → clientes(id)
	tipo	VARCHAR(80)	NOT NULL
	marca/modelo	VARCHAR(80/100)	
equipamentos	num_serie	VARCHAR(100)	
	id	BIGINT	PK, AUTO_INCREMENT
	nome	VARCHAR(150)	NOT NULL
	preco	DECIMAL(10,2)	NOT NULL, CHECK >= 0
	qtd_estoque	INT	NOT NULL, DEFAULT 0
produtos	estoque_min	INT	DEFAULT 5
	id	BIGINT	PK, AUTO_INCREMENT
	data	DATETIME	NOT NULL, DEFAULT NOW()
	valor_total	DECIMAL(10,2)	NOT NULL
	cliente_id	BIGINT	FK → clientes(id)
vendas	usuario_id	BIGINT	FK → usuarios(id)
	id	BIGINT	PK, AUTO_INCREMENT
	venda_id	BIGINT	FK → vendas(id), CASCADE
	produto_id	BIGINT	FK → produtos(id)
	quantidade	INT	NOT NULL, CHECK > 0
itens_venda	preco_unitario	DECIMAL(10,2)	NOT NULL

9. Repositório GitHub

Link: github.com/<seu-usuario>/sgc-techfix

Caminho	Conteúdo
README.md	Descrição, tecnologias e instruções de execução
backend/	Código-fonte Spring Boot
docs/diagramas/	Diagramas de Classes, Domínio e Banco (.png)
docs/SGC_Entrega1.pdf	Este documento
sql/script_criacao.sql	Script SQL de criação do banco
.gitignore	Ignorar target/, .env, *.class, *.jar

Padrão de commits sugerido

- feat: adiciona entidade Cliente
- docs: adiciona diagrama de classes
- chore: configura projeto Spring Boot inicial
- sql: adiciona script de criação do banco

Checklist da Entrega 1

Item	Critério	Status
Descrição do sistema	Contexto real descrito (TechFix)	Incluso
Requisitos funcionais	10 RFs definidos e detalhados	Incluso
Requisitos não funcionais	8 RNFs com categorias	Incluso
Arquitetura em camadas	6 camadas descritas com tecnologias	Incluso
Design Patterns	Repository + DTO + Singleton justificados	Incluso
Diagrama de Domínio	Entidades e relacionamentos	Pendente diagrama visual
Diagrama de Classes	Classes, pacotes e membros	Pendente diagrama visual
Diagrama lógico do banco	Tabelas, tipos e chaves	Pendente diagrama visual
Script SQL	script_criacao.sql	Arquivo separado
Repositório GitHub	Estrutura e commits iniciais	Criar e commitar